

Controlled AI/ML Production Engineering: Making AI Workflows Reproducible, Reversible and Traceable

Iain J. Clark

Document Controls

Version: 1.0

9 April 2026

Status:

Working Draft

Overview

This note sets out practical software engineering principles for moving AI/ML workflows from exploratory development toward controlled production systems. As data scientists, we take ownership of models throughout their life cycle¹. This requires that we can answer questions about models trained months earlier, now operating on datasets that may have evolved since training. In some cases, a candidate model may demonstrate instability, drift or leakage² and need to be rolled back to its predecessor. These are important life-cycle considerations, often less formalized in standard data science practice than in traditional software engineering. Without these controls, models cannot be reliably reproduced, investigated internally, or audited, introducing both operational and governance risk. This document is general professional guidance only; all examples are illustrative and do not reflect any production system, employer-specific implementation, confidential information, or internal process.

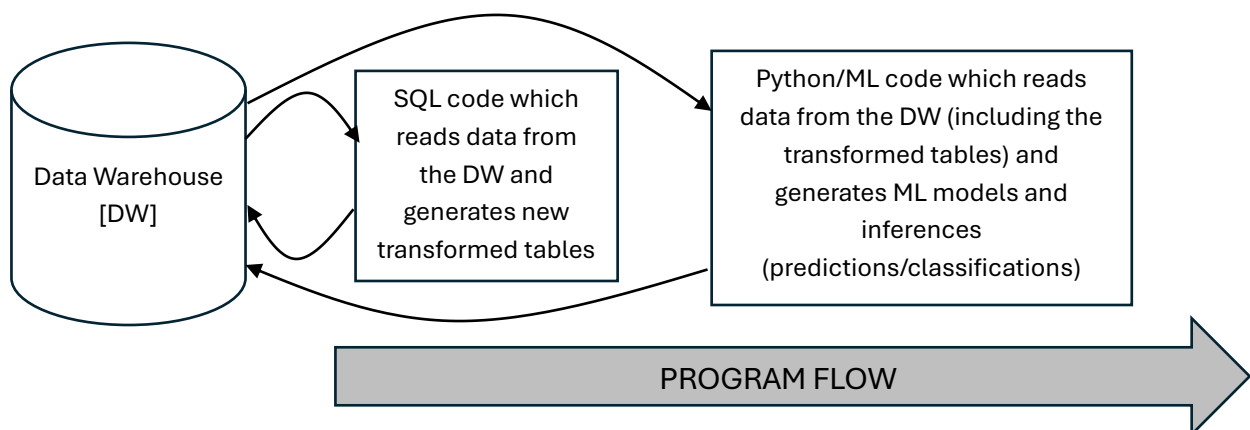
1. Introduction

This note sets out a practical framework for implementing the machine learning operations (MLOps) process described in [1], wherein MLOps is defined as “a set of standardized processes and technology capabilities for building, deploying, and operationalizing ML systems rapidly and reliably”. Echoing this definition, I seek to give equal weighting to “rapidly” and “reliably”.

Industry-wide, SQL (Structured Query Language) and Python are frequently used to build descriptive, predictive and prescriptive systems, which are often hosted in a modern cloud data platform (e.g. Databricks, Snowflake), although in-house (“on-prem”) hosting is possible.

This framework is platform-agnostic and can be implemented using a range of modern data platforms. It reflects one possible synthesis of established practices across industry.

Functionally, the workflow looks like this:



¹ e.g. Development, Test, QA (Quality Assurance), UAT (User Acceptance Testing), and Production environments.

² This framework includes Risk Registers which account for these possible defects. We don't pretend our code is bug-free, though we try to make it so. However, if defects are found, we have a mechanism for controlled rollback.

Many data platforms provide model registry capabilities. Once persisted, these models can be loaded and used for ongoing inference, performing tasks such as classification and prediction.

How do we use the Python/DW interoperability to set up a framework for full lifecycle management, monitoring, and rollback provisions for our AI models?

2. Operating Principles

A useful principle is that horizon dates should be specified at the start of any codebase, in **one** place. For example,

```
SET TRAINING_DATE = CURRENT_DATE(); -- SQL
```

or

```
training_date = date.today() # Python
```

All subsequent logic uses this training date variable, with no hidden dependencies on calls to system date functions. This permits **reproducibility** – we can run workflows again in the future by modifying them to use the historical training date that the model was trained on.

Code pertaining to workflows e.g. SQL or Python code should be saved in a version control system such as git³, which associates a unique identifier (a SHA) with each committed version.

How do we know what training date was used for a particular model run? The answer is simple: we save that metadata within a model registry.

```
model_version = register_model(model)
metadata = { "code_version": git_sha,
            "training_date": training_date,
            "metrics": metrics
          }
```

```
attach_metadata(model_version, metadata)
```

This persists not just the model, but the git SHA corresponding to the Python code used to generate the model, the training date it was generated on, and the performance metrics collected during the training/calibration of the model – in that order, in the pseudocode fragment above.

This permits **reversibility** and **traceability** – we can revert⁴ to any earlier version of a saved model *and* its archived code, permitting reproducibility for any organisation⁵ using this framework.

References

[1] Salama, K., Kazmierczak, J., & Schut, D. (2021). *Practitioners guide to MLOps: A framework for continuous delivery and automation of machine learning* [White paper]. Google Cloud.

https://services.google.com/fh/files/misc/practitioners_guide_to_mlops_whitepaper.pdf

³ Git is a widely used version control system for tracking changes to software source code, allowing specific committed versions of code to be retrieved later. <https://git-scm.com/>

⁴ e.g., for audit, investigation, or controlled rollback.

⁵ “All examples are illustrative and do not reflect any production system or employer-specific implementation.